

Rechnen mit Fehlern

Ayleen Peschke

12. Mai 2026

Gliederung

1. Einführung
2. Binäre Suche
3. Fehlerfortpflanzung
4. Kondition
5. Fehleranalyse
6. Newton-Verfahren

Übersicht

1. Einführung

1.1 Allgemeines

1.2 Datenfehler

1.3 Rundungsfehler

1.4 Verfahrensfehler

1.5 Darstellungen von Fehlern

2. Binäre Suche

3. Fehlerfortpflanzung

4. Kondition

5. Fehleranalyse

6. Newton-Verfahren

Allgemeines

- ▶ Bei numerischen Berechnungsproblemen muss immer mit einem Fehler gerechnet werden
- ▶ Diese entstehen z.B. durch eine fehlerhafte Eingabe
- ▶ Es wird zwischen drei Arten von Fehlern unterschieden

Datenfehler

- ▶ Der Rechner benötigt zu Beginn Eingabedaten
- ▶ Die Maschinenzahl ist jedoch begrenzt
- ▶ Eingabedaten sind häufig Messdaten, diese sind oft gestört
- ▶ $\tilde{x}$ beschreibt den gestörten Eingabewert zum korrekten Wert x

Rundungsfehler

- ▶ Der Maschinenzahlbereich ist nur begrenzt
- ▶ Alle Zwischenergebnisse müssen gerundet werden

Verfahrensfehler

- ▶ Ein Algorithmus ist eine (theoretisch) unendlich lange Folge
- ▶ Die Folge konvergiert gegen das gewünschte Ergebnis
- ▶ Nach endlich vielen Schritten wird das Verfahren abgebrochen

Modellfehler

- ▶ Das mathematische Modell beschreibt das Problem nicht immer genau
- ▶ Dabei entstehen die Modellfehler

Darstellungen von Fehlern

- ▶ Der absolute Fehler $\Delta x := |x - \tilde{x}|$ beschreibt die Abweichung des gestörten Wertes zum eigentlichen Wert.
- ▶ Der relative Fehler $\varepsilon := \frac{\Delta x}{x}$ beschreibt den Anteil des absoluten Fehlers vom exakten Wert.

Übersicht

1. Einführung

2. Binäre Suche

2.1 Definition

2.2 Algorithmus

3. Fehlerfortpflanzung

4. Kondition

5. Fehleranalyse

6. Newton-Verfahren

Binäre Suche

Definition 2.1

Sei $f : \mathbb{R} \rightarrow \mathbb{R}$ eine monoton wachsende Funktion und sei $y \in \mathbb{R}$. Existieren die Schranken $L, U \in \mathbb{R}$ mit $f(L) \leq y \leq f(U)$, so lässt sich $f^{-1}(y)$ durch die binäre Suche berechnen:

Sei $m_i = \frac{l_i + u_i}{2}$ der Mittelpunkt des Intervalls $[l_i, u_i]$:

Gilt $f(m_i) < y$, setze $m_i = l_{i+1}$ und fahre mit dem Intervall $[l_{i+1}, u_i]$ fort und betrachte $f(m_{i+1})$.

Gilt $f(m_i) > y$, setze $m_i = u_{i+1}$ und fahre mit dem Intervall $[l_i, u_{i+1}]$ fort und betrachte $f(m_{i+1})$.

Für die Folge $(m_n)_{n \in \mathbb{N}}$ aller Mittelpunkte gilt $\lim_{n \rightarrow \infty} m_n = f^{-1}(y)$. Man muss die Funktion $f(x)$ nicht kennen, sondern diese nur auswerten können. Man sagt, f ist durch ein Orakel gegeben.

Beispiel

Beispiel 2.2

Wir wählen $y = 3$, $f(x) = x^2$ und unsere Schranken mit $L = 1$ und $U = 2$, da $\sqrt{3} \in [1, 2]$:

$$1,5^2 = 2,25 \Rightarrow \sqrt{3} \in [1,5, 2]$$

$$1,75^2 = 3,0625 \Rightarrow \sqrt{3} \in [1,5, 1,75]$$

$$1,625^2 = 2,640625 \Rightarrow \sqrt{3} \in [1,625, 1,75]$$

$$1,6875^2 = 2,84765625 \Rightarrow \sqrt{3} \in [1,6875, 1,75]$$

$$1,7134375^2 = 2,9541015625 \Rightarrow \sqrt{3} \in [1,71875, 1,75]$$

$$1,734375^2 = 3,008056640625 \Rightarrow \sqrt{3} \in [1,71875, 1,734375]$$

Definition 2.3

Sei $[l_i, u_i]$ das zu betrachtende Intervall und $m_i = \frac{l_i + u_i}{2}$ dessen Mittelpunkt in der i -ten Iteration. Es gilt

$|m_i - \sqrt{3}| \leq \frac{1}{2} \cdot |u_i - l_i| = 2^{-i} |u_1 - l_1| = 2^{-i} |U - L|$ Diese Schranke wird *a-priori-Schranke* genannt.

Im Nachhinein lässt sich eine genauere Schranke angeben, zum

Beispiel $|m_i - \sqrt{3}| = \frac{|m_i^2 - 3|}{m_i + \sqrt{3}} \leq \frac{|m_i^2 - 3|}{m_i + l_i}$. Diese Schranke nennt man *a-posteriori-Schranke*.

Algorithmus

Algorithmus 2.4

Eingabe: Ein Orakel zur Berechnung einer monoton wachsenden

Funktion $f : \mathbb{Z} \rightarrow \mathbb{R}$, $L, U \in \mathbb{Z}$ mit $L \leq U$, $y \in \mathbb{R}$ mit $y \geq f(L)$

Ausgabe: das maximale $n \in \{L, \dots, U\}$ mit $f(n) \leq y$

$l \leftarrow L$

$u \leftarrow U + 1$

while $u > l + 1$ **do**

$m \leftarrow \left\lfloor \frac{l + u}{2} \right\rfloor$

if $f(m) > y$ **then** $u \leftarrow m$ **else** $l \leftarrow m$

output l

Satz 2.5

Der Algorithmus 2.4 arbeitet korrekt und terminiert nach $O(\log(U - L + 2))$ Iterationen.

Beweis zu Satz 2.5

Beweis.

Es gilt jederzeit $L \leq l \leq u - 1 \leq U$ und $f(l) \leq y$, sowie $u > U$ und $f(u) > y$. Also liegt das korrekte Ergebnis immer in $\{l, \dots, u - 1\}$.

Außerdem gilt

$$\begin{aligned} & \max \left\{ \left\lfloor \frac{l+u}{2} \right\rfloor - l - 1, u - \left\lfloor \frac{l+u}{2} \right\rfloor - 1 \right\} \\ & \leq \max \left\{ \frac{l+u}{2} - l - 1, u - \frac{l+u-1}{2} - 1 \right\} \\ & \leq \frac{u-l-1}{2}, \end{aligned}$$

das heißt $u - l - 1$ wird mit jeder Iteration mindestens halbiert und die Laufzeit beträgt $O(\log(U - L + 2))$ Iterationen. $\square$

Übersicht

1. Einführung
2. Binäre Suche
3. Fehlerfortpflanzung
4. Kondition
5. Fehleranalyse
6. Newton-Verfahren

Lemma 3.1

Seien $x, y \in \mathbb{R} \setminus \{0\}$ und $\tilde{x}, \tilde{y} \in \mathbb{R}$ Näherungen mit $\varepsilon_x := \frac{x-\tilde{x}}{x}$ und $\varepsilon_y := \frac{y-\tilde{y}}{y}$. Dann gilt für $\varepsilon_o := \frac{x \circ y - (\tilde{x} \circ \tilde{y})}{x \circ y}$ mit $\varepsilon \in \{+, -, \cdot, /\}$:

$$\varepsilon_+ = \varepsilon_x \cdot \frac{x}{x+y} + \varepsilon_y \cdot \frac{y}{x+y},$$

$$\varepsilon_- = \varepsilon_x \cdot \frac{x}{x-y} - \varepsilon_y \cdot \frac{y}{x-y},$$

$$\varepsilon_\cdot = \varepsilon_x + \varepsilon_y - \varepsilon_x \cdot \varepsilon_y,$$

$$\varepsilon_/ = \frac{\varepsilon_x - \varepsilon_y}{1 - \varepsilon_y}$$

Beweis zu Lemma 3.1

Beweis.

Es gilt $\varepsilon_x = \frac{x - \tilde{x}}{x} \Leftrightarrow \tilde{x} = x \cdot (1 - \varepsilon_x)$ und analog $\tilde{y} = y \cdot (1 - \varepsilon_y)$

$$\begin{aligned}\varepsilon_+ &= \frac{x + y - (\tilde{x} + \tilde{y})}{x + y} = \frac{x + y - (1 - \varepsilon_x) \cdot x - (1 - \varepsilon_y) \cdot y}{x + y} \\ &= \varepsilon_x \cdot \frac{x}{x + y} + \varepsilon_y \cdot \frac{y}{x + y},\end{aligned}$$

ε_- : *erfolgt analog*



Beweis zu Lemma 3.1

Beweis.

$$\begin{aligned}\varepsilon. &= \frac{x \cdot y - \tilde{x} \cdot \tilde{y}}{x \cdot y} = \frac{x \cdot y - (1 - \varepsilon_x) \cdot x \cdot (1 - \varepsilon_y) \cdot y}{x \cdot y} \\ &= 1 - (1 - \varepsilon_x) \cdot (1 - \varepsilon_y) = \varepsilon_x + \varepsilon_y - \varepsilon_x \cdot \varepsilon_y\end{aligned}$$



Beweis.

$$\begin{aligned}\varepsilon_I &= \frac{x/y - \tilde{x}/\tilde{y}}{x/y} = \frac{x/y - ((1 - \varepsilon_x) \cdot x)/((1 - \varepsilon_y) \cdot y)}{x/y} \\ &= 1 - \frac{1 - \varepsilon_x}{1 - \varepsilon_y} = \frac{1 - \varepsilon_y - 1 + \varepsilon_x}{1 - \varepsilon_y} = \frac{\varepsilon_x - \varepsilon_y}{1 - \varepsilon_y}\end{aligned}$$



Übersicht

1. Einführung

2. Binäre Suche

3. Fehlerfortpflanzung

4. **Kondition**

4.1 Definition

4.2 Beispiel

5. Fehleranalyse

6. Newton-Verfahren

Kondition

Definition 4.1

Sei $P \subseteq D \times E$ ein numerisches Berechnungsproblem mit $D, E \subseteq \mathbb{R}$. Sei $d \in D$ eine Instanz von P mit $d \neq 0$.

Wir definieren die **(relative) Kondition** $k(d)$ von d als

$$\limsup_{\varepsilon \rightarrow 0} \left\{ \inf \left\{ \frac{\left| \frac{e-e'}{e} \right|}{\left| \frac{d-d'}{d} \right|} \mid e \in E, (d, e) \in P \right. \right. \\ \left. \left. \mid d' \in D, e' \in E, (d', e') \in P, 0 < |d - d'| < \varepsilon \right\} \right\}$$

wobei wir hier die Konvention $\frac{0}{0} = 0$ verwenden.

Ist $k(d)$ die Kondition der Instanz $d \in D$, so ist die **(relative) Kondition** des Problems P

$$\sup \{k(d) \mid d \in D, d \neq 0\}.$$

Proposition 4.2

Sei $f : D \rightarrow E$ ein eindeutiges numerisches Berechnungsproblem mit $D, E \subseteq \mathbb{R}$, und sei $d \in D$ eine Instanz mit $d \neq 0$ und $f(d) \neq 0$. Dann ist ihre Kondition gleich

$$\frac{|d|}{|f(d)|} \cdot \limsup_{\varepsilon \rightarrow 0} \left\{ \frac{|f(d') - f(d)|}{|d' - d|} \mid d' \in D, 0 < |d' - d| < \varepsilon \right\}$$

Korollar 4.3

Sei $f : D \rightarrow E$ ein eindeutig numerisches Berechnungsproblem, wobei $D, E \subseteq \mathbb{R}$ und f differenzierbar sei. Dann ist die Kondition einer Instanz $d \in D$ mit $d \neq 0$ und $f(d) \neq 0$

$$k(d) = \frac{|f'(d)| \cdot |d|}{|f(d)|}$$

Beispiel 4.4

- ▶ Sei $f(x) = \sqrt{x}$. Dann ist die Kondition $k(d) = \frac{\frac{1}{2\sqrt{x}} \cdot x}{\sqrt{x}} = \frac{1}{2}$. Das heißt die Wurzelfunktion ist gut konditioniert.
- ▶ Sei $f(x) = x + a$ und es gilt $k(x) = \left| \frac{x}{x+a} \right|$. Ist $|x + a| < |x|$, so ist die Funktion schlecht konditioniert.
- ▶ Sei $f(x) = a \cdot x$. Diese Funktion ist immer gut konditioniert, da $k(x) = \frac{|a| \cdot |x|}{|x|} = 1$

Übersicht

1. Einführung

2. Binäre Suche

3. Fehlerfortpflanzung

4. Kondition

5. Fehleranalyse

5.1 Arten der Fehleranalyse

5.2 Beispiel

6. Newton-Verfahren

Numerisch stabil

Ein Algorithmus heißt numerisch stabil, wenn alle Rechenschritte gut konditioniert sind. Numerisch stabil ist, genau wie gut konditioniert, nicht exakt definiert.

Arten der Fehleranalyse

- ▶ **Vorwärtsanalyse:** Es wird untersucht, wie sich der relative Fehler im Laufe einer Rechnung entwickelt.
- ▶ **Rückwärtsanalyse:** Jedes Zwischenergebnis wird als exaktes Ergebnis gewertet und es wird abgeschätzt, wie groß die Eingangsdatenstörung dafür sein müsste.

Beispiel 5.1

- ▶ *Vorwärtsanalyse:* $x \oplus y = \text{rd}(x + y) = (x + y) \cdot (1 + \varepsilon)$ mit $|\varepsilon| \leq \text{eps}(F)$
- ▶ *Rückwärtsanalyse:* $x \oplus y = x \cdot (1 + \varepsilon) + y \cdot (1 + \varepsilon) = \tilde{x} + \tilde{y}$ mit $|\varepsilon| \leq \text{eps}(F)$

eps(F) beschreibt hierbei den größtmöglichen Rundungsfehler der Maschine.

Beispiel

Wir betrachten erneut $\sqrt{3}$.

Für 1,734375 ist die korrekte Eingabe

$$1,734375^2 = 3,008056640625 = 3(1 + \varepsilon) \text{ mit } \varepsilon \leq 0,00269.$$

Da die Kondition $\frac{1}{2}$ beträgt, kann der Fehler durch $\frac{1}{2} \cdot 0,00269$ beschränkt werden.

Übersicht

1. Einführung
2. Binäre Suche
3. Fehlerfortpflanzung
4. Kondition
5. Fehleranalyse
6. Newton-Verfahren
 - 6.1 Beispiel
 - 6.2 Konvergenzordnung

Newton-Verfahren

- ▶ Beschreibt ein weiteres Näherungsverfahren
- ▶ Eignet sich oft besser als die binäre Suche
- ▶ Gesucht ist die Nullstelle einer differenzierbaren Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ mit $f' \neq 0$

$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)}, \quad n = 0, 1, \dots$$

- ▶ Dabei wird ein „geratener“ Startwert x_0 gewählt

Beispiel

Beispiel 6.1

Um $\sqrt{3}$ zu berechnen, wähle $f(x) = x^2 - a$, wobei $a = 3$. Es gilt:

$$x_{n+1} \leftarrow x_n - \frac{x_n^2 - a}{2x_n} = \frac{1}{2} \cdot \left(x_n + \frac{a}{x_n} \right), n = 0, 1, \dots$$

Wir starten mit $x_0 = 1$ und erhalten

$$x_1 = \frac{1}{2} \cdot \left(1 + \frac{3}{1} \right) = 2$$

$$x_2 = \frac{1}{2} \cdot \left(2 + \frac{3}{2} \right) = \frac{7}{4} = 1,75$$

$$x_3 = 1,732142857\dots$$

$$x_4 = 1,732050810\dots$$

Konvergenzordnung

Definition 6.2

Sei $(x_n)_{n \in \mathbb{N}}$ eine konvergente Folge reeller Zahlen und $x^* := \lim_{n \rightarrow \infty} x_n$. Existiert eine Konstante $c < 1$ und ein $n_0 \in \mathbb{N}$, verringert sich der relative Fehler, so dass

$$|x_{n+1} - x^*| \leq c \cdot |x_n - x^*|$$

für alle $n \geq n_0$ gilt, so hat die Folge (mindestens) Konvergenzordnung 1.

Sei $p > 1$. Existiert eine Konstante $c \in \mathbb{R}$, so dass

$$|x_{n+1} - x^*| \leq c \cdot |x_n - x^*|^p$$

für alle $n \in \mathbb{N}$ gilt, so hat die Folge (mindestens) Konvergenzordnung p .

Eine Folge konvergiert linear bzw. quadratisch, falls die Konvergenzordnung 1 bzw. 2 ist.

Konvergenzordnung

Satz 6.3

Sei $a \geq 1$ und $x_0 \geq 1$. Die Folge $(x_n)_{n \in \mathbb{N}}$ ist definiert durch

$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$ für $n = 0, 1, \dots$. Für diese Folge gilt:

- (a) $x_1 \geq x_2 \geq \dots \geq \sqrt{a}$
- (b) Die Folge x_n konvergiert quadratisch gegen $\lim_{n \rightarrow \infty} x_n = \sqrt{a}$.
Genauer gilt für alle $n \geq 0$:

$$x_{n+1} - \sqrt{a} \leq \frac{1}{2}(x_n - \sqrt{a})^2$$

- (c) Für alle $n \geq 1$ gilt:

$$x_n - \sqrt{a} \leq x_n - \frac{a}{x_n} \leq 2(x_n - \sqrt{a})$$

Beweis Satz 6.3 (a)

Beweis.

- (a) Nach dem Satz vom arithmetischen und geometrischen Mittel gilt $\sqrt{xy} \leq \frac{x+y}{2}$. Daraus folgt

$$\sqrt{a} = \sqrt{x_n + \frac{a}{x_n}} \leq \frac{1}{2} \left(x_n + \frac{a}{x_n} \right) = x_{n+1}$$

Außerdem gilt für $n \geq 1$:

$$x_{n+1} - x_n = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right) - x_n = \frac{x_n^2}{2x_n} + \frac{a}{2x_n} - \frac{2x_n^2}{2x_n} = \frac{1}{2x_n} (a - x_n^2) \leq 0,$$

da $\sqrt{a} \leq x_n$ äquivalent ist zu $a \leq x_n^2$. Daraus folgt $x_{n+1} \leq x_n$,
woraus die Behauptung folgt.



Beweis Satz 6.3 (b)

Beweis.

(b) Für alle $n \geq 0$ ist:

$$x_{n+1} - \sqrt{a} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right) - \sqrt{a} = \frac{1}{2x_n} (x_n^2 + a - 2x_n\sqrt{a}).$$

Es gilt $\frac{1}{x_n}(x_n - \sqrt{a}) \leq 1$ und daraus folgt

$x_{n+1} - \sqrt{a} \leq \frac{1}{2}(x_n - \sqrt{a})$ für $n \geq 1$. Daraus folgt

$\lim_{n \rightarrow \infty} x_n = \sqrt{a}$ beweist, da sich der Fehler mit jedem Schritt mindestens halbiert. Aus $x_n \geq 1$ folgt die Behauptung.



Beweis Satz 6.3 (c)

Beweis.

(c) Für $n \in \mathbb{N}$ gilt:

$$x_n - \sqrt{a} \stackrel{(a)}{\leq} x_n - \frac{a}{x_n} = 2x_n - 2x_{n+1} \stackrel{(a)}{\leq} 2(x_n - \sqrt{a})$$



Vielen Dank für eure Aufmerksamkeit!¹

¹Der Vortrag hierzu basiert auf: [Stefan Hougardy und Jens Vygen](#). *Algorithmische Mathematik*. 2., korrigierte und erweiterte Auflage. Berlin: Springer Spektrum, 2018.

Literaturverzeichnis



Hougardy, Stefan und Jens Vygen. *Algorithmische Mathematik*.
2., korrigierte und erweiterte Auflage. Berlin: Springer
Spektrum, 2018.